Tool Design

> Part of the MCP Server curriculum. This document covers designing tool schemas for an LLM audience: direct wraps vs. composed tools, argument-resolution guards, shaping responses for a model rather than a human, error handling, statelessness, interpretive framing, envelope pass-through with computed summaries, and rejecting what a tool cannot honor.

1. A tool's real audience is a model, not a compiler

A tool's input schema is type-checked, the same as any typed function signature — but the name, description, and parameter documentation are doing real work that no type system checks. A model decides **whether** and **how** to call a tool by reading exactly that text, much like a person decides whether to click a labeled button by reading its label — a confusingly-named or under-described tool doesn't throw an error, it just goes unused, or worse, gets called with the wrong arguments, silently. Writing tool descriptions is closer to writing UX copy than to writing an internal function's docstring: assume the reader (the model) has no other context than what's in front of it right now.

2. Direct wraps vs. composed tools

Style	What it is	When it fits
Direct wrap	One tool maps ~1:1 onto one underlying data operation (a single API endpoint, a single query)	The underlying API is already shaped around individual, well-scoped questions — the common case when the API itself was designed resource-by-resource (see API Design's REST conventions)
Composed	One tool internally makes several calls, or does real aggregation/ranking logic, to answer one higher-level question a user is actually likely to ask, that no single underlying endpoint answers directly	The underlying API has no single operation shaped like the question (e.g. "who are the top N right now" when the only endpoint returns *all* rows for *every* entity, unranked and unfiltered)

A composed tool is often the difference between one tool call and many: if answering "give me the forecasts for the top 10 emitters" naively required calling a single-entity forecast tool once per entity, a model faced with that gap will do exactly that — issue the same tool call ten times in a row, burning ten round trips (and, if each round trip is a separate LLM reasoning step, ten times the latency and cost) to answer one question. A composed tool that fans the underlying calls out **internally and concurrently**, returning one aggregated result, collapses this into a single call from the model's perspective. This is a genuinely common pattern worth watching for: if real usage shows a natural, frequently-asked question consistently triggers a burst of near-identical sequential tool calls, that's a strong signal a composed tool is missing, not that the model is being inefficient.

3. Argument-resolution guards

Free-text identifiers (an entity name a model might type slightly wrong, abbreviate, or refer to by an alias) need an explicit policy for what happens on a near-miss — leaving it to whatever the underlying data source happens to do by default is a real risk, because many data sources are built to silently drop or empty-result an unmatched value, which is a *worse* failure mode for a model caller than for a human one. A human looking at a suddenly-empty chart notices something's wrong; a model can just narrate "there is no data for that" with complete confidence and never suspect its own input had a typo.

A layered resolution guard, sitting in front of every tool argument shaped like a free-text identifier:

1. **Exact match** against the canonical set of valid values first.
2. **Fuzzy match above a high similarity threshold** — auto-resolve to the closest match without asking, since above that threshold a mismatch is essentially always a typo, not a genuinely different intended value.
3. **Below that threshold**, return an explicit, structured tool error naming the nearest candidates (`"No match for 'Congo' — did you mean: Congo, Democratic Republic of Congo?"`) so the model has a concrete, correctable next step, rather than guessing silently or returning an empty result with no explanation.
4. A distinct fourth case, easy to miss: a value that's **valid in general but out of scope for this specific tool or its current arguments** (a real, recognized entity that this particular tool simply doesn't cover) — this deserves its own clear message pointing at what scope *would* include it, rather than being folded into the same bucket as "not recognized at all."

4. Shaping responses for a model, not a human's eyeballs

- **Cap and annotate, don't silently truncate.** If an unscoped or default query could return an unbounded number of rows, cap the response and add an explicit note field stating what was shown, how many exist in total, and by what ordering — a model repeating an unannotated partial result as if it were complete is a direct, high-visibility failure mode a human reader would notice immediately from a table's row count, but a model has no equivalent instinct unless the data itself says so.
- **The presence (or absence) of that note field is itself part of the contract.** Design it so "no note" reliably means "this is everything" — that lets the system prompt tell the model to only mention scope/trimming when the field is present, rather than reasoning about it from row counts every time.
- **Trigger trimming on "no explicit selection given," not on "returns more than one thing."** If the caller explicitly asked for a specific list of entities, return exactly those, uncapped — trimming an explicit request would silently drop something the caller asked for by name, which is a different and worse failure than trimming an open-ended default query.
- **Return structured data, never a UI directive.** A tool's job ends at "here is the answer, structured"; deciding *how* to render that answer is a separate, later concern (see the conversational-agent curriculum's generative-UI section) — keeping that decision out of the tool itself is what lets the same tool's output be rendered differently by different consumers, or re-rendered differently later, without touching the tool at all.

5. Error handling and statelessness

- Prefer a **structured tool error** (the `isError: true` result shape from Core Concepts, §7) over either letting an exception propagate uncaught (which can crash the whole server process, taking down every other tool

with it) or swallowing the error into a misleadingly "successful" empty response (which looks to the model exactly like "there is genuinely no data," the same trap as an unannotated silent drop in §4).
- Keep a data-serving server **stateless across calls** wherever possible — the same statelessness principle that applies to a REST API applies here for the same reasons: a server that just re-reads or re-queries its source on every call is simpler to run, restart, and scale than one juggling per-session state, and any request can be served by any running instance. Add caching only once real call volume demonstrably justifies the added complexity of invalidation — not by default (mirrors the API backend curriculum's own caching discussion).

6. Grounding: give the model a way to explain itself

When the underlying analysis has a real, nontrivial methodology behind it (how a number was computed, what a category boundary means, what a model's known limitations are), it's worth exposing that as its own tool (or resource) returning **static, canonical, pre-written text** — sourced from one shared location rather than re-derived per response — so the model can *quote* documented methodology when a user asks "how was this computed" instead of improvising an explanation that might be subtly wrong or drift from wording used elsewhere in the system.

6.1 Put the interpretive framing in the tool itself

Grounding is not only a separate "methodology" tool. When a tool returns analysis whose *meaning* is easy to overstate — a statistical relationship, a scenario translation, a preliminary dataset — state what it is **not** in the tool's own description ("a simplified data-driven relationship, not a climate model; correlation, not proof of cause; no attribution to any single actor"). The description travels with the tool to *every* client, so a desktop LLM app that has never heard of your agent's system prompt still gets the framing. Fix naming the same way: if two analyses are easy to confuse and only one of them is allowed to carry a particular established name, say in the descriptions, summaries and notes which one may be called that and which may not.

6.2 Keep the source's envelope, and add a computed summary

For a family of tools wrapping a governed data domain, two conventions pay for themselves:

1. **Pass the source envelope through verbatim.** If the upstream response carries a note, caveats, attribution/licence text and a source vintage, keep all of it in the tool result unmodified. Downstream code (an agent's required-disclosure logic) depends on those fields being there; a tool that "tidies" them away breaks that dependency silently.
2. **Add a deterministic `summary` object computed in code** — first/last value and year, change, slope with its confidence interval where a fit is present, latest reading, the unit. The model should quote `summary`, never derive a figure from a raw series. Return the full series alongside it (a chart needs it) and leave *capping what the model sees* to the consuming agent, so the server stays consumer-agnostic.

6.3 Reject what you cannot honor; never silently reinterpret

The resolution guard in §3 is one instance of a broader rule: when an argument cannot be honored as given, say so.

- **An explicit empty list is not "use the default."** `countries=[]` means "none of them," which is a caller bug; silently substituting the default selection answers a question nobody asked. Reject it, in every tool that takes a list (a missed tool is a hole — check them all).
- **Arguments that belong to a different mode are errors, not noise.** If a tool has two modes (a time series for named entities vs. a ranking), reject an argument from the wrong mode rather than dropping it; the model otherwise believes its filter was applied.
- **An unsupported combination names the supported ones.** The upstream's "422: unsupported pair" becomes a tool error that lists the valid combinations — never a quiet substitution.
- **Translate identifiers when the upstream wants different ones.** If an endpoint takes codes rather than display names, the tool resolves the model's text against *that endpoint's own* entity set (using the same exact → fuzzy → explicit-error pattern) instead of reusing a list that belongs to a different endpoint.
- **Defaults should be the useful answer.** "Top N" with no year should mean the latest year *with data*, not the latest calendar year (which may be empty), and a result that depends on how many entities were ranked should return that count so a consumer never assumes it.
- **Say whether each returned entity is representable downstream.** If a consuming UI can only show a subset of entities, flag each series in the result (`in_expanded_scope: true`) so a link-builder can carry only what the page can show — see the agent curriculum's follow-up links.
- **Normalise user-visible text in one place.** Strings built from upstream responses (methodology, notes) should pass through one function that applies the house typography (proper unit subscripts and symbols, en dashes) — otherwise the first string nobody remembered leaks the raw form to users.

7. Naming and description conventions

- Name tools as `verb_noun` (`get_top_emitters`, not `topEmittersQuery` or `TE`) — a consistent, guessable pattern lets a model (and a developer) predict a tool's shape from its neighbors.
- Write descriptions the way you'd explain the tool out loud to a new colleague, not the way you'd write an internal comment — spell out units, formats, and any non-obvious default ("returns metric tons, not the raw dataset's original unit"; "years outside 1990–2023 are not covered").
- For any parameter with a fixed, small set of valid values, constrain it at the schema level (an enum/`Literal` type, mirroring FastAPI's own approach) rather than describing the valid values only in prose — a model is far less likely to hallucinate an invalid value when the schema itself makes the invalid value simply not expressible.

8. Tool-design checklist

Concern	Do	Avoid
Naming	`verb_noun`, consistent across the tool set	Ambiguous or cutely-abbreviated names
Ambiguous identifiers	Layered resolution guard (§3), explicit error with candidates	Silent drop or empty result on a near-miss
Large/default responses	Cap + explicit annotation field (§4)	Silent truncation

Concern	Do	Avoid
Failure	Structured `isError` result (§5)	Uncaught exceptions; misleadingly "successful" empty results
State	Stateless by default (§5)	Per-session caching added preemptively
Rendering	Structured data only (§4)	Returning a UI directive or generated markup
Framing	What the tool is *not*, in its own description (§6.1)	Relying on one client's system prompt to supply the caveat
Envelope & figures	Pass the source envelope through; add a computed `summary` (§6.2)	Dropping caveats; making the model derive numbers
Unhonorable input	Reject: empty lists, wrong-mode args, unsupported pairs (§6.3)	Silently substituting a default or dropping the argument

See also

- MCP Server curriculum index
- Core Concepts, §7 for the `isError` shape referenced in §5
- Testing — verifying the guards and shaping rules in this document actually behave as designed
- Conversational Agents, Tool Calling and Guardrails for how a consuming agent builds on the guarantees a well-designed tool set provides